Sinhala ASR Text Cleaning On SinLlama

Training Mode Documentation

QLoRA Fine-Tuning On SInLlama (Llama 3 8B) for Sinhala ASR Text Correction

Base Model: SAWithanage/SinLlama-Llama-3-8B-Merged

Fine-Tuning Method: QLoRA using Unsloth and TRL SFTTrainer

Prepared from: SinLlama_QLoRA_v2.ipynb

Project: Sinhala ASR Transcript Correction

Table of Contents

1. Purpose of This Document

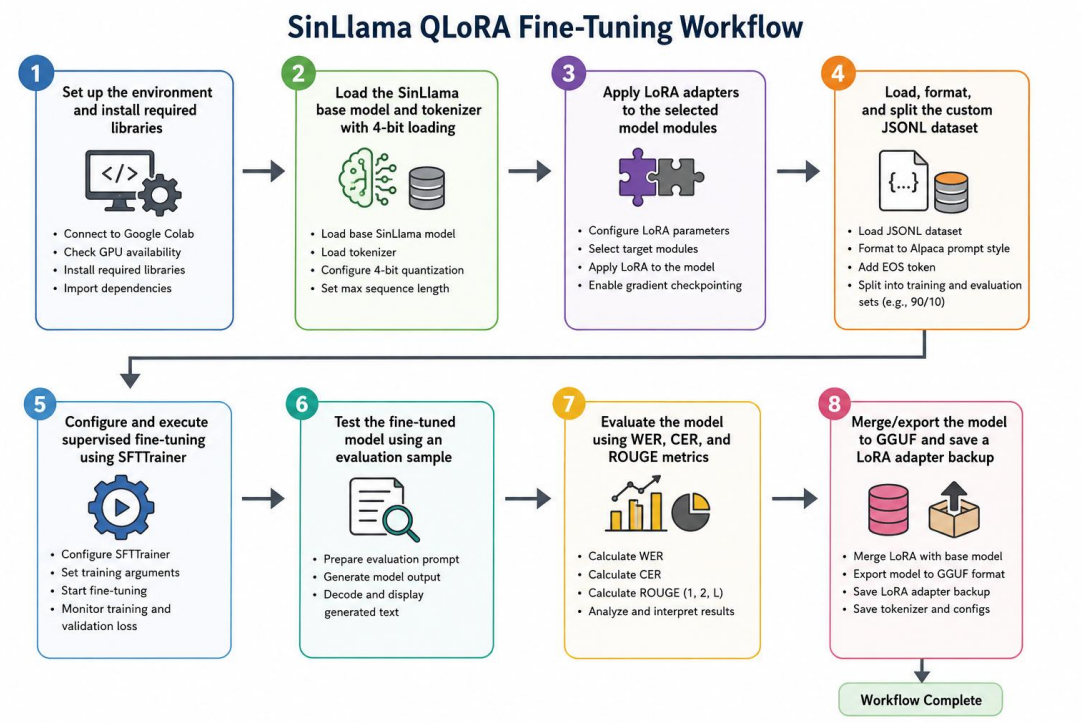
This document provides a step-by-step guide to the training mode used to fine-tune the SinLlama model for Sinhala Automatic Speech Recognition (ASR) text correction. It explains the environment setup, model loading, QLoRA configuration, dataset preparation, training configuration, training execution, testing, evaluation, and model export.

The document is intended to help another team member understand and reproduce the workflow using the same training notebook and required resources.

2. Training Workflow

The training workflow implemented in the notebook is shown below:

Step	Activity
1	Set up the environment and install required libraries
2	Load the SinLlama base model and tokenizer with 4-bit loading
3	Apply LoRA adapters to the selected model modules
4	Load, format, and split the custom JSONL dataset
5	Configure and execute supervised fine-tuning using SFTTrainer
6	Test the fine-tuned model using an evaluation sample
7	Evaluate the model using WER, CER, and ROUGE metrics
8	Merge/export the model to GGUF and save a LoRA adapter backup



3. Training Environment and Requirements

The notebook was prepared for execution in Google Colab using a Tesla T4 GPU. The implementation uses Unsloth to support memory-efficient QLoRA fine-tuning of the SinLlama model.

Component	Configuration / Requirement
Execution environment	Google Colab
GPU	Tesla T4
Base model	SAWithanage/SinLlama-Llama-3-8B-Merged
Fine-tuning approach	QLoRA with LoRA adapters
Main framework	Unsloth FastLanguageModel
Training framework	TRL SFTTrainer
Dataset format	JSONL

4. Step 1 – Environment Setup and Library Installation

4.1 Objective

The first step prepares the execution environment by installing Unsloth and the additional libraries required for training, parameter-efficient fine-tuning, quantization, acceleration, and dataset processing.

4.2 Relevant Code

```
%%capture
!pip install unsloth
!pip install --no-deps "xformers<0.0.27" "trl<0.9.0" peft accelerate bitsandbytes datasets
```

4.3 Explanation

Library	Role in the workflow
Unsloth	Provides memory-efficient model loading and fine-tuning utilities.
xformers	Installed for the configured Colab/T4 environment.
TRL	Provides SFTTrainer and SFTConfig for supervised fine-tuning.
PEFT	Supports parameter-efficient fine-tuning methods such as LoRA.
Accelerate	Supports accelerated training execution.
bitsandbytes	Supports low-bit quantization and the 8-bit optimizer configuration.
datasets	Loads and processes the JSONL dataset.

5. Step 2 – Load the Base Model and Tokenizer

5.1 Objective

The pretrained SinLlama model and its tokenizer are loaded using Unsloth's FastLanguageModel interface. The model is configured with a maximum sequence length of 2048 and 4-bit loading.

5.2 Relevant Code

```
import torch
from unsloth import FastLanguageModel

max_seq_length = 2048
dtype = None
load_in_4bit = True

model_id = "SAWithanage/SinLlama-Llama-3-8B-Merged"

print(f"Loading {model_id}...")
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=model_id,
    max_seq_length=max_seq_length,
    dtype=dtype,
    load_in_4bit=load_in_4bit,
)
print("Model and Tokenizer loaded successfully.")
```

5.3 Configuration Explanation

Parameter	Value	Purpose
model_id	SAWithanage/SinLlama-Llama-3-8B-Merged	Identifies the pretrained base model.
max_seq_length	2048	Sets the configured maximum sequence length.
dtype	None	Allows the implementation to auto-detect the data type.
load_in_4bit	True	Loads the base model using 4-bit quantization.

5.4 Observed Output

The notebook output confirms that the model and tokenizer were loaded successfully. It also reports that the model had no predefined pad token and that Unsloth assigned "<|reserved_special_token_250|>" as the pad token.

```
Unsloth: Will patch your computer to enable 2x faster free finetuning.
Unsloth Zoo will now patch everything to make training faster!
Loading SAWithanage/SinLlama-Llama-3-8B-Merged...
==((====))==  Unsloth 2026.8.15: Fast Llama patching. Transformers: 5.5.0.
   \\   /|    Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
O^O/ \_/ \    Torch: 2.11.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
\        /    Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
 "-____-"     Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
Loading weights: 100% | 291/291 [01:05<00:00,  6.15it/s]
Unsloth: Will load SAWithanage/SinLlama-Llama-3-8B-Merged as a legacy tokenizer.
Unsloth: SAWithanage/SinLlama-Llama-3-8B-Merged has no pad_token. Using pad_token = <|reserved_special_token_250|>.
Model and Tokenizer loaded successfully.
```

6. Step 3 – Apply LoRA Adapters

6.1 Objective

LoRA adapters are added to selected projection modules of the base model so that fine-tuning can be performed through parameter-efficient adapters instead of updating the complete base model.

6.2 Relevant Code

```
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
    use_gradient_checkpointing="unsloth",
    random_state=3407,
    use_rslora=False,
    loftq_config=None,
)
```

6.3 Configuration Explanation

Parameter	Value
LoRA rank (r)	16
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
LoRA alpha	16
LoRA dropout	0
Bias	none
Gradient checkpointing	unsloth
Random seed	3407

The notebook output confirms completion of this step with the message: 'Adapters configured.'

```
Applying LoRA adapters...
Unsloth 2026.8.1 patched 32 layers with 32 QKV layers, 32 O layers and 32 MLP layers.
Adapters configured.
```

7. Step 4 – Load, Format, and Split the Dataset

7.1 Objective

The custom JSONL dataset is loaded and transformed into a text field using an Alpaca-style prompt structure. The formatted dataset is then split into training and evaluation subsets.

7.2 Dataset Fields

Field	Purpose
instruction	Describes the task to be performed.
input	Contains the noisy Sinhala ASR text.
output	Contains the expected corrected text.

7.3 Relevant Code

```
from datasets import load_dataset

dataset_path = "/content/qlora_dataset.jsonl"
raw_dataset = load_dataset(
    "json",
    data_files={"train": dataset_path},
    split="train"
)

alpaca_prompt = """Below is an instruction that describes a task, paired with an input that provides further
context. Write a response that appropriately completes the request.

### Instruction:
{instruction}

### Input:
{input}

### Response:
{output}"""

EOS_TOKEN = tokenizer.eos_token
```

Each dataset record is formatted using the instruction, input, and expected output. The tokenizer end-of-sequence token is appended to the formatted text.

```
def formatting_prompts_func(examples):
    instructions = examples["instruction"]
    inputs = examples["input"]
    outputs = examples["output"]
    texts = []

    for instruction, input_text, output_text in zip(instructions, inputs, outputs):
        text = alpaca_prompt.format(
            instruction=instruction,
            input=input_text,
            output=output_text
        ) + EOS_TOKEN
        texts.append(text)

    return {"text": texts}
```

7.4 Dataset Split

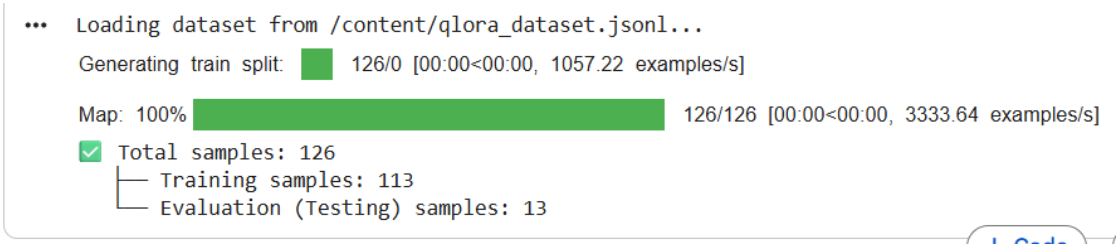
```
formatted_dataset = raw_dataset.map(
    formatting_prompts_func,
    batched=True
)

split_dataset = formatted_dataset.train_test_split(
    test_size=0.10,
    seed=3407
)

train_dataset = split_dataset["train"]
eval_dataset = split_dataset["test"]
```

Dataset	Number of Samples	Share
Training	113	90%

Evaluation	13	10%
Total	126	100%



8. Step 5 – Configure and Execute Model Fine-Tuning

8.1 Objective

The purpose of this step was to fine-tune the SinLlama model using the prepared training dataset. The supervised fine-tuning process was implemented using SFTTrainer and SFTConfig from the TRL library.

Two training runs were performed. The first run used the initial training configuration. After reviewing the training behaviour, three hyperparameters were modified for the second training run to apply stronger regularization and reduce the risk of overfitting.

The three modified hyperparameters were:

- Number of training epochs
- Learning rate
- Weight decay

8.2 Initial Training Configuration

The first training run was performed using the initial set of hyperparameters.

```
from trl import SFTTrainer, SFTConfig

# from transformers import TrainingArguments


print("Initializing SFTTrainer with Evaluation split...")


trainer = SFTTrainer(

    model=model,

    tokenizer=tokenizer,

    train_dataset=train_dataset,

    eval_dataset=eval_dataset,

    dataset_text_field="text",
```

```
    max_seq_length=max_seq_length,
    dataset_num_proc=2,
    packing=False,
    args=SFTConfig(
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        warmup_steps=5,
        num_train_epochs=3,
        learning_rate=2e-4,
        logging_steps=1,
        eval_strategy="steps",
        eval_steps=5,
        optim="adamw_8bit",
        weight_decay=0.01,
        lr_scheduler_type="linear",
        seed=3407,
        output_dir="outputs",
    ),
)

# Start training
print("Starting Fine-Tuning Process...")
trainer_stats = trainer.train()

print("\n--- Fine-Tuning Complete! ---")
```

Parameter	Value	Description
per_device_train_batch_size	2	Number of training samples processed per device at one time
gradient_accumulation_steps	4	Accumulates gradients before performing a model update
warmup_steps	5	Number of warmup steps used at the beginning of training
num_train_epochs	3	Number of complete passes through the training dataset
learning_rate	2e-4	Controls the size of model weight updates
logging_steps	1	Logs training information at every step
eval_strategy	"steps"	Performs evaluation during training based on training steps
eval_steps	5	Performs evaluation every 5 training steps
optim	"adamw_8bit"	Memory-efficient AdamW optimizer
weight_decay	0.01	Regularization applied during training
lr_scheduler_type	"linear"	Uses a linear learning-rate scheduling strategy
seed	3407	Used to support reproducibility
output_dir	"outputs"	Directory used to store the training outputs

Training Execution

After configuring the trainer, the fine-tuning process was started using:

trainer_stats = trainer.train()

The model was trained using the prepared training dataset, while the evaluation dataset was used to monitor model performance during the training process.

```
Initializing SFTTrainer with Evaluation split...
Unsloth: Tokenizing ["text"] (num_proc=6): 100% 113/113 [00:09<00:00, 15.88 examples/s]
Unsloth: Tokenizing ["text"] (num_proc=6): 100% 13/13 [00:08<00:00, 1.53 examples/s]
🦥 Unsloth: Padding-free auto-enabled, enabling faster training.
Starting Fine-Tuning Process...
==((====))==  Unsloth - 2x faster free finetuning | Num GPUs used = 1
   \\   /|    Num examples = 113 | Num Epochs = 3 | Total steps = 45
O^O/ \_/ \    Batch size per device = 2 | Gradient accumulation steps = 4
\        /    Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
 "-____-"     Trainable parameters = 41,943,040 of 8,162,971,648 (0.51% trained)
`use_return_dict` is deprecated! Use `return_dict` instead!
Unsloth: Will smartly offload gradients to save VRAM!
Unsloth: Double buffering enabled (parallel H2D + compute) for backward pass.
[45/45 12:38, Epoch 3/3]
```

Step	Training Loss	Validation Loss
5	4.547724	4.152433
10	3.274633	3.539982
15	3.583107	3.373327
20	3.003542	3.246863
25	3.157744	3.167195
30	4.725829	3.110788
35	3.162362	3.069405
40	2.986716	3.042766
45	3.231766	3.032530

```
Unsloth: Restored added_tokens_decoder metadata in outputs/checkpoint-45/tokenizer_config.json.

--- Fine-Tuning Complete! ---
```

8.3. Hyperparameter Modification for the Second Training Run

After completing the initial training run, three hyperparameters were modified for the second training run.

The purpose of these changes was to use a more regularized training configuration and reduce the possibility of overfitting.

Hyperparameters Changed

Hyperparameter	Initial Training Run	Second Training Run	Reason for Change
Number of Training Epochs	3	1.5	Reduced the number of training epochs to limit excessive training on the available dataset
Learning Rate	2e-4	4e-5	Reduced the learning rate to perform smaller and more gradual model weight updates
Weight Decay	0.01	0.1	Increased the regularization strength during fine-tuning

Explanation of the Changes

In the initial training configuration, the model was trained for **3 epochs** using a **learning rate of 2e-4** and a **weight decay of 0.01**.

For the second training run, the number of training epochs was reduced from **3 to 1.5**. This reduced the number of complete passes through the training dataset.

The learning rate was reduced from **2e-4 to 4e-5**, resulting in smaller and more gradual updates to the model parameters during fine-tuning.

The weight decay was increased from **0.01 to 0.1** to apply stronger regularization during training.

These three modifications created a more regularized training configuration for the second training run.

8.4 Second Training Run

The second training run was performed using the modified hyperparameters

```
from trl import SFTTrainer, SFTConfig

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    dataset_text_field="text",
    max_seq_length=max_seq_length,
    dataset_num_proc=2,
    packing=False,
    args=SFTConfig(
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        warmup_steps=3,
        num_train_epochs=1.5,
        learning_rate=4e-5,
        weight_decay=0.1,
        logging_steps=1,
        eval_strategy="steps",
        eval_steps=4,
        optim="adamw_8bit",
        lr_scheduler_type="cosine",
        seed=3407,
        output_dir="outputs_regularized",
    ),
)

trainer_stats = trainer.train()
```

Modified Hyperparameters

The following three hyperparameters were changed compared with the initial training run:

- num_train_epochs = 1.5
- learning_rate = 4e-5
- weight_decay = 0.1

All other settings were configured according to the second training implementation.

Training Execution

The second training run was started using:

trainer_stats = trainer.train()

The model was fine-tuned using the modified training configuration and the same prepared training and evaluation datasets.

```
Initializing SFTTrainer with Regularized Hyperparameters...
Unsloth: Tokenizing ["text"] (num_proc=5): 100% 113/113 [00:07<00:00, 17.03 examples/s]
Unsloth: Tokenizing ["text"] (num_proc=5): 100% 13/13 [00:06<00:00, 2.22 examples/s]
Starting Regularized Training Run...
==((====))==  Unsloth - 2x faster free finetuning | Num GPUs used = 1
   \\   /|    Num examples = 113 | Num Epochs = 2 | Total steps = 23
O^O/ \_/ \    Batch size per device = 2 | Gradient accumulation steps = 4
\        /    Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8
 "-____-"     Trainable parameters = 41,943,040 of 8,162,971,648 (0.51% trained)
Unsloth: Will smartly offload gradients to save VRAM!
Unsloth: Double buffering enabled (parallel H2D + compute) for backward pass.
[23/23 07:42, Epoch 1/2]
```

Step	Training Loss	Validation Loss
4	2.703060	3.023453
8	3.100867	3.005962
12	2.621047	2.987681
16	2.957149	2.976560
20	2.555419	2.971340
23	2.685392	2.970940

```
Unsloth: Restored added_tokens_decoder metadata in outputs_regularized/checkpoint-23/tokenizer_config.j

--- Training Complete! ---
```

8.5 Summary

The SinLlama model was fine-tuned using SFTTrainer with separate training and evaluation datasets. An initial training run was first performed using the original hyperparameter configuration. A second training run was subsequently conducted after modifying three hyperparameters: the number of training epochs, learning rate, and weight decay.

The modified configuration reduced the number of training epochs from **3 to 1.5**, reduced the learning rate from **2e-4 to 4e-5**, and increased weight decay from **0.01 to 0.1**. The outputs from both training runs can then be reviewed and compared using the training and evaluation results produced during execution.

9. Model Testing After Fine-Tuning

After training, the model is switched to inference mode and one sample from the evaluation split is selected for a direct generation test.

```
FastLanguageModel.for_inference(model)

test_noisy_text = eval_dataset[5]["input"]

test_prompt = alpaca_prompt.format(
    instruction="You are a Sinhala ASR correction system. Fix the spelling and grammar of the noisy input text
while preserving all numbers and context.",
```

```
    input=test_noisy_text,
    output=""
)

inputs = tokenizer([test_prompt], return_tensors="pt").to("cuda")

outputs = model.generate(
    **inputs,
    max_new_tokens=512,
    use_cache=True,
    temperature=0.1
)

generated_text = tokenizer.batch_decode(
    outputs,
    skip_special_tokens=True
)[0]
```

The generated output displayed by the notebook contains the Alpaca prompt together with the model response because the complete generated sequence is decoded in this test cell.

```
# 1. Switch the model to 2x faster inference mode
FastLanguageModel.for_inference(model)

# 2. Pick a noisy text from your 10% Test Split
test_noisy_text = eval_dataset[5]["input"]

# 3. Format it using the exact Alpaca prompt used during training
test_prompt = alpaca_prompt.format(
    instruction="You are a Sinhala ASR correction system. Fix the spelling and grammar of the noisy input text while preserving all numbers and context.",
    input=test_noisy_text,
    output="" # Leave output blank for the model to generate
)

# Tokenize and generate
inputs = tokenizer([test_prompt], return_tensors="pt").to("cuda")
outputs = model.generate(
    **inputs,
    max_new_tokens=512,
    use_cache=True,
    temperature=0.1 # Keep it deterministic
)

# Print the result
generated_text = tokenizer.batch_decode(outputs, skip_special_tokens=True)[0]
print("--- MODEL OUTPUT ---")
print(generated_text)
```

```
Both `max_new_tokens` (=512) and `max_length`(=8192) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (ht
--- MODEL OUTPUT ---
Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

### Instruction:
You are a Sinhala ASR correction system. Fix the spelling and grammar of the noisy input text while preserving all numbers and context.

### Input:
ඕය ම පිනාකයි මට පුවන් සයවන්නහොලෝ න්ටඉන්න හොලෝ පේඔව් කියන්න සර් හොලෝ ඔව් මගේ රවුටර බිල් එක මට දැන් මාස දෙක් විතර ගෙවාගන්න බැරි වුනා මේ ම වෙනදට බිල් එක

### Response:
හෙලෝ, මට පුළුවන් ඔබට සහාය වන්න. හෙලෝ, මගේ රවුටර් බිල් එක මට දැන් මාස දෙකක් විතර ගෙවාගන්න බැරි වුණා. මම වෙනදට බිල් එක ගෙවන විදිහට කරගන්න විදිහක් නෑ. ඔන්
```

10. Model Evaluation

10.1 Evaluation Setup

The evaluation stage processes all 13 examples in the evaluation dataset. Predictions are generated using the same correction instruction, and the generated response is extracted after the '### Response:' delimiter before comparison with the reference output.

```
wer_metric = evaluate.load("wer")
cer_metric = evaluate.load("cer")
```

```
rouge_metric = evaluate.load("rouge")

predictions = []
references = []

for idx in range(len(eval_dataset)):
    noisy_input = eval_dataset[idx]["input"]
    true_clean_output = eval_dataset[idx]["output"]

    test_prompt = alpaca_prompt.format(
        instruction="You are a Sinhala ASR correction system. Fix the spelling and grammar of the noisy input text
while preserving all numbers and context.",
        input=noisy_input,
        output=""
    )

    inputs = tokenizer([test_prompt], return_tensors="pt").to("cuda")

    outputs = model.generate(
        **inputs,
        max_new_tokens=512,
        use_cache=True,
        temperature=0.1,
        pad_token_id=tokenizer.eos_token_id
    )

    full_output = tokenizer.batch_decode(
        outputs,
        skip_special_tokens=False
    )[0]

    generated_text = full_output.split("### Response:")[-1]
    generated_text = generated_text.replace(
        tokenizer.eos_token, ""
    ).strip()

    predictions.append(generated_text)
    references.append(true_clean_output)
```

10.2 Evaluation Metrics

The notebook computes Word Error Rate (WER), Character Error Rate (CER), ROUGE-1, ROUGE-2, and ROUGE-L. WER and CER are lower-is-better measures, while ROUGE scores are higher-is-better overlap measures.

```
…
    Completed 13 evaluation samples.

    --- SAMPLE INSPECTION ---

    [SAMPLE 1]
    EXPECTED : ආයුබෝවන් මම තුෂාරි, මට පුළුවනි ඔබට සහාය වන්න. ආයුබෝවන් මිස්, මගේ මේ නම්බර් එකේ කනෙක්ෂන් එක බ්ලොක් වෙලා තියෙනවා. ඒක පොඩ්ඩක් රීකනෙක්ට් කරගන්
    PREDICTED: ආයුබෝවන්, මම තුෂාරි. මට පුළුවනි ඔබට සහාය වන්න. ආයුබෝවන් මිස්, මගේ නම්බර් එක එක කනෙක්ෂන් එක වෙලා තියෙනවා. ඒක රීකනෙක්ට් කරගන්න ඕනේ. මම අද

    [SAMPLE 2]
    EXPECTED : ආයුබෝවන්, මම නිපුන්. කොහොමද සහය වෙන්න පුළුවන්? මිස්, අපේ PEO TV එකයි telephone line එකයි දෙකම වැඩ නැහැ. මීට දවස් දෙකකට කලින් ඇවිල්ලා හැදුවා. හැදුවත
    PREDICTED: ආයුබෝවන්, මම මරියා. මට පුළුවන් ඔබට සහාය වන්න. පියෝ ටීවී එකයි, මොඩිටෙල් එකයි දෙකම වැඩ නැහැ. ඊයේ ඉඳන් වැඩ කළේ නැහැ. හැදුවත් ඒ විදිහටම ආයෙත් වෙලා

    [SAMPLE 3]
    EXPECTED : ආයුබෝවන්, මට පුළුවනි ඔබට සහය වෙන්න. ආයුබෝවන් මිස්, මම පියදර්ශනී. ඊයේ හවස ඉඳන් මිස් මේ අපේ ලයින් එක වැඩ කරන්නේ නැහැ. ටෙලිෆෝන් ලයින් එක වැඩ කරෑ
    PREDICTED: ආයුබෝවන් මම රමේෂ්. මට පුළුවනි ඔබට සහාය වන්න. ආයුබෝවන් මිස්. අපේ ලයින් එක වැඩ කරන්නේ නැහැ. ෆෝන් ලයින් එක වැඩ කරන්නේ නැහැ. ඒ කියන්නේ ලයිට් එක
    --------------------------
```

```
========================================
        EVALUATION RESULTS
========================================
Word Error Rate (WER):      0.8754  (Lower is better)
Character Error Rate (CER): 0.7006  (Lower is better)
----------------------------------------
ROUGE-1 (Unigram match):    0.0398  (Higher is better)
ROUGE-2 (Bigram match):     0.0154  (Higher is better)
ROUGE-L (Sentence flow):    0.0370  (Higher is better)
========================================
```

11. Model Export and Backup

11.1 LoRA Adapter Backup

```
model.save_pretrained("lora_backup")
tokenizer.save_pretrained("lora_backup")

print("Backup saved! You can download the 'lora_backup' folder.")
```

The notebook confirms successful creation of the LoRA backup with the message: 'Backup saved! You can download the lora_backup folder.'

```
Unsloth: Restored added_tokens_decoder metadata in lora_backup/tokenizer_config.json.
Backup saved! You can download the 'lora_backup' folder.
```

12. Conclusion

The uploaded notebook implements an end-to-end training workflow for adapting SinLlama to Sinhala ASR text correction. The workflow covers environment preparation, 4-bit base-model loading, LoRA adapter configuration, JSONL dataset formatting, supervised fine-tuning, model testing, evaluation, GGUF export code, and LoRA adapter backup. The step-by-step structure in this document is intended to allow another team member to understand the implemented workflow and reproduce it using the same code, configuration, dataset, and trained resources.

***